

Understanding Knowledge-Intensive Processes: from Traces to Instance Graphs

Claudia Diamantini, Laura Genga, Domenico Potena

Department of Information Engineering

Università Politecnica delle Marche

Ancona, Italy

{c.diamantini, l.genga, d.potena}@univpm.it

Abstract— Enterprise information systems, while support daily activities, typically collect data on executed processes in event logs. These data describe the temporal sequence in which activities were carried out, hiding possible parallelism and other control flows. Representing the structure of each process execution in the form of an Instance Graph, enables managers to discover valuable knowledge on enterprise behaviors. In this work, we describe BIG4ProM, a tool which implements the Building Instance Graph (BIG) algorithm. BIG4ProM exploits filtering Process Discovery algorithms implemented in ProM in order to return the set of instance graphs related to the given event log. The plug-in is conceived to support both expert and standard users.

Keywords—process mining; building instance graph; knowledge-intensive process; ProM

I. INTRODUCTION

Nowadays enterprise information systems, like Enterprise Resource Planning (ERP), Customer Relationship Management (CRM), Business Process Management Systems (BPMS), and Workflow Management Systems (WMS), store data on business processes that are daily executed. These data are typically recorded in an *event log*, where each element of the log (i.e., a *trace*) describes the temporal sequence in which activities were carried out, and usually the actors which performed each activity. Depending on the system, other information can be found as well, like the executor, input/output data, and the total duration of the activity (or, equivalently, its end time). Hence, although a single execution of a business process (called *process instance*, or *case*) can usually include parallel executions and cycles, in the event log the process is flattened, losing its structure.

Analyzing information stored in event logs enables managers to discover valuable knowledge on enterprise behaviors, in order to monitor and improve operations performance. Simple analyses can be performed directly on sequential traces. For instance, average instance lead time, resource usage, event interval analysis [1] can be obtained. However, more insights can be derived only when the control flow structure of the instance is known or, in other terms, when the model from which the instance is generated is known, namely the *instance model*. An instance model can be represented as a graph, where nodes are activities and arcs are

causal relations between activities. Such a graph is called *Instance Graph* (IG); Figures 2 and 3 show examples of IGs.

The building of IGs is driven by the knowledge of causal relations among activities, which either is a-priori held or can be inferred from event logs. Process Discovery is the discipline devoted to infer relations from a log and build models. Process Discovery (PD) techniques belong to the broader Process Mining discipline, whose goal consists in discovering, monitoring and improving a given process exploiting event log generated during process execution [2]. Most of techniques aim to generate a complete process model. However, such a model is not always required, and a good representation of each individual process instance can provide useful insights as well, and can be used in addressing several tasks, e.g., [3, 4, 5, 6, 7]. In [3] instance models are used in conformance checking applications. [4] uses instance models to implement a log delta-analysis technique, which allows to identify relevant differences between the executions recorded in two event logs. Another relevant application is described in [5, 6, 7], where a methodology is proposed aimed at extracting the most relevant common substructures in a set of instance models.

In the literature, few approaches have been proposed to building IGs from simple event logs, namely logs providing the minimal set of information (i.e., timestamp, actor and activity), and without resorting to any domain knowledge. In this setting, to the best of our knowledge, only three approaches have been proposed. [4] and [8] are based on the causal relations inference rules embedded in the α -algorithm PD technique. This technique perform fairly well with well-structured processes, but they show limitations when applied to unstructured, knowledge-intensive processes. This kind of processes is typical of real-world domains, where human actors have high degree of freedom in the execution of activities. This is the case, for instance, of health care where, although clinical guidelines are given, there may be good reasons to deviate from such guidelines [9, 10]. The limits of the application of α -algorithm to knowledge-intensive processes are well studied in the literature. They are related to the inference of models affected by overgeneralization. In practice, being unable to recognize existing causal relations among activities, the algorithm generates models involving a high number of parallel activities, thus allowing for more instances than the real ones.

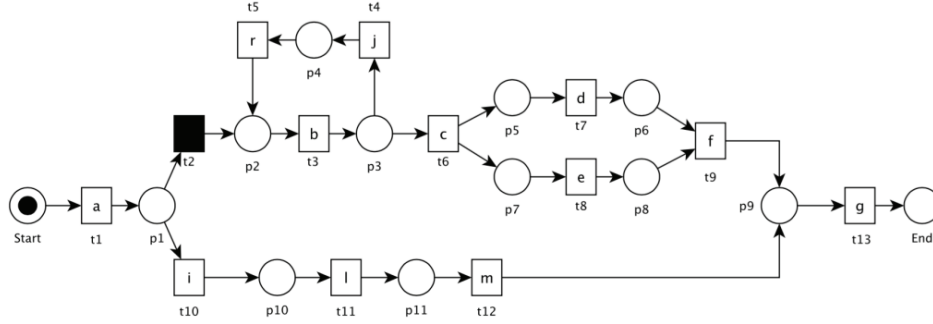


Figure 1: A Petri net

Similarly, the use of the algorithm to build IG produce poor results, in terms of inaccurate, overgeneralizing IG, accounting for much more traces than those actually recorded in the event log.

For these reasons, in a previous work, we proposed the Building Instance Graph (BIG) algorithm, which is specifically conceived for unstructured processes [11]. In particular, the BIG algorithm uses causal relation inference rules typical of *filtering* techniques, which model only the most frequent process behaviors, thus being able to cut down overgeneralization. However, the application of a filtering technique also presents some drawbacks. In particular, (causal relations of) several traces of the event log are now not represented in the model (i.e., they are irregular with respect to the model), impacting negatively on the quality of the corresponding IG. While in loosely variable domains this can still be acceptable, since irregular traces can be considered the result of “wrong” or outlier behaviors, and can then be discarded, this is not the case with highly variable processes, where no execution can be really targeted as wrong and then overlooked, or in the case we like to analyze exactly those outlier behaviors. To address this issue, in BIG the exploitation of filtering techniques is coupled with an original technique aimed at repairing IG of irregular traces, thus providing the analyst with a more complete and faithful knowledge of the reality registered in the event log.

In this work, we describe BIG4ProM, a tool which implements the BIG algorithm and has been developed as a plug-in of the Process Mining Framework (ProM), a widely adopted open source tool, which provides an extensible platform for the use and the development of process mining algorithms [13]. BIG4ProM exploits filtering PD algorithms implemented in ProM in order to return the set of instance graphs related to the given event log. The plug-in is conceived to support both expert and standard users, from the selection of the PD technique to the visualization of resulting IGs.

II. THE BIG ALGORITHM

Given an event log, the outcome of the BIG algorithm consists in a set of instance graphs, one for each trace stored in the log. The algorithm involves two main steps, i.e. an *IG*

extraction step and an *IG repairing step*. In the extraction step, each trace of the event log is converted in an IG following the definition provided in [4]. More precisely, given a trace l_i , its corresponding IG_i is $IG_i = (V_i, E_i, \Phi_i)$ where: V_i is the set of the nodes, each corresponding to an event in the trace; E_i is the set of the edges, each linking a pair of events (e_i, e_j) such that i) a causal relation exists between the corresponding activities, and ii) neither causal successors of e_i nor causal predecessors of e_j exist in between the two events in the trace; and Φ_i is a labelling function, which associates each node with the name of the activity of the corresponding event. The problem with definition proposed in [4] is that it does not take into account the possible presence of noise and/or “irregular” behaviors, representing deviations from the expected/desired process execution.

When applied to this kind of event log, [4] returns overgeneralizing, highly imprecise IGs, which provide little or no support to explore the activities execution flow. As an example, let us consider the trace $l_{del} = \langle a, b, c, d, e, g, \rangle$ and let us build the corresponding IG using the causal relations shown by the Petri net in Figure 1. Note that l_{del} does not fit this model, since the activity f is missing (i.e., there has been a deletion). The IG corresponding to l_{del} is shown in Figure 2. As we can see, we obtain a disconnected graph, which does not allow to understand in which portion of the process the activity g is supposed to be executed. Such a representation is clearly undesired. Similar issues can occur in case of inserted events, i.e. events occurred in positions not allowed by the process model.

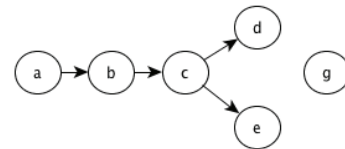


Figure 2: IG_{del}

To address these issues, the BIG algorithm exploits a *repairing procedure*, aimed at manipulating the IGs to enhance their precision. To this end, first we need to recognize which

are the anomalous traces in the event log. This can be done by means of a *conformance checking* technique, i.e. a technique devoted to check whether or not an event log fits with a given process model [12]. These techniques allow us to identify for each irregular IGs the positions and the kinds of the anomalies. Then, we apply tailored rules for repairing deleted and inserted events. For deleted events, the repairing consists in identifying events which should have been connected to the deleted event and properly connecting them. For the insertion repairing, we have to change the edges connecting the nodes corresponding to the event(s) before and the event(s) after the inserted event, to connect such nodes with the node corresponding to the inserted event in the graph. Figure 3 shows the repaired IG for I_{del} . More details on the BIG algorithm and examples are provided in [11].

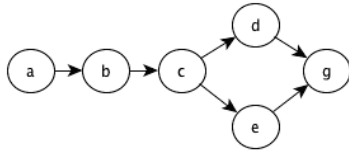


Figure 3: repaired IG_{del}

III. BIG4PROM

We have implemented the BIG algorithm above delineated as a plug-in of the Process Mining Framework (ProM). Due to its plug-in architecture, the implementation of new algorithms and their integration within the framework is quite straight.

Figure 4 shows the starting window of the plug-in.



Figure 4: starting windows of BIG4Prom plugin variants

As usual for ProM plug-ins, in the left side are listed the required inputs, while in the right side there are the expected outputs. As we can note, there are two variants of the plug-in; the first one, that is the one highlighted in the figure, requires an event log, a Petri net and the results of a Conformance Checking technique. The other variant, listed immediately below the first one, requires only the event log. In both cases, the output consists in an Instance Graphs set. In the following we discuss the second variant, that is intended for the standard users, while the other is more intended for expert users. As the user started the plug-in, a configuration window is displayed, as shown in Figure 5.

The user has to select if she intends to build repaired or not repaired graphs, by checking/unchecking the “Repair” checkbox, respectively. Note without the repairing, the user obtains in outcome IGs built by applying the IG definition provided in [8]. The other parameter regards the technique to use to derive the causal relation set. Currently, three Process Discovery techniques can be selected, i.e. the α -algorithm [14], the Heuristic Miner [15] and the infrequent Inductive Miner [16].

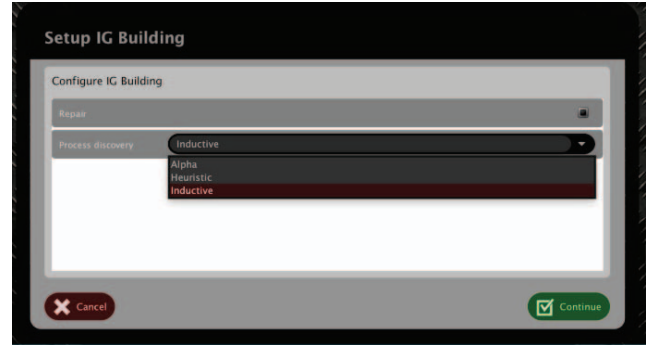
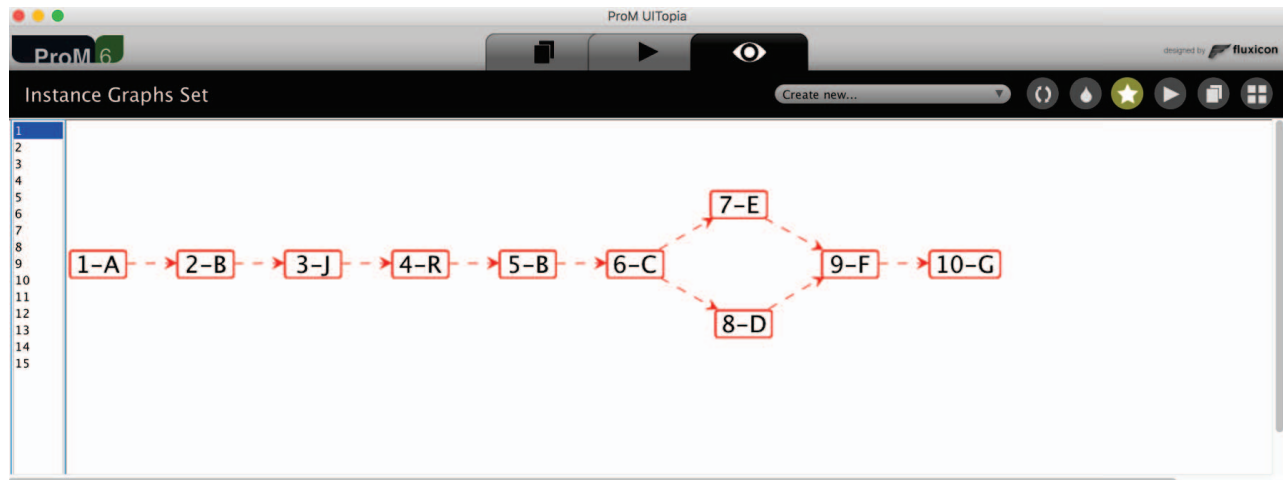


Figure 5: starting windows of BIG4Prom plugin variants

On the basis of the user's choice, the ProM plug-in implementing the selected technique is invoked, using its default configuration. Note that in order to repair the graph, the BIG algorithm also requires to test the conformance of the event log with respect to the mined process model. To this end, our plug-in exploits the *PNetReplayer* plug-in, implementing the conformance checking technique presented in [12]. Note that the call to the *PNetReplayer* is performed automatically by the plug-in, without requiring any user's action.

The variant of the plug-in discussed so far makes most of the IGs building process transparent for the user. Indeed, the user has only to load the event log to analyze, without configuring neither possible parameters of Process Discovery techniques, nor the conformance checking plug-in. However, some users (e.g., expert users) can require more freedom in selecting Process Discovery or conformance checking techniques, as well as in parameters configuration. These users can exploit the other variant of the BIG4Prom plugin, which allows them to use a their own process model and conformance checking results. In this regard, it should be noted that, despite our approach is not constrained to a specific Process Discovery technique, the current implementation of our plug-in requires to use Petri nets as process model formalism. Furthermore, our plug-in expects the results of the conformance checking to have the same format of the *PNetReplayer* outcome. Therefore, if the user intends to use a different conformance checking technique, she has to check that the outcome fits the expected format. We would like to point out that using this variant the user has only to set if repair or not repair the graphs; hence, the configuration window is simpler than the one shown in Figure 5, i.e. it shows only the “Repairing” checkbox.

Figure 6 shows an example of an Instance Graphs set visualized by ProM. User can select the graph to visualize by using the left panel, which lists all the IGs ids, corresponding to the order of the corresponding trace in the event log.



The Figure 6: Instance Graphs visualization

For example, the IG with id 1 corresponds to the first trace in the log. By selecting an IG id, in the central panel it is displayed the corresponding IG. For each node the plugin displays the name of the activity and its position in the corresponding trace. The user can adjust the position of the nodes on the basis of her needs. Besides visualizing the graphs set, the plugin also exports each graph as a single dot file. This is useful in order to visualize and use the generated IGs outside ProM.

REFERENCES

- [1] S. Suriadi, C. Ouyang, W. van der Aalst, and A.ter Hofstede, "Event interval analysis: Why do processes take time?", *Decision Support Systems*, 79, pp 77-98, 2015.
- [2] W. van der Aalst, *Process Mining: Discovery, Conformance and Enhancement of Business Processes*. Springer, 2011.
- [3] X. Lu, D. Fahland, and W. van der Aalst, "Conformance checking based on partially ordered event data", in: F. Fournier, J. Mendling (Eds.), *Business Process Management Workshops, LNBIP*, vol 202, pp. 75-88. Springer International Publishing, 2015.
- [4] N. van Beest, M. Dumas, L. Garcia-Banuelos, and M. La Rosa, "Log delta analysis: Interpretable differencing of business process event logs", in: H. R. Motahari-Nezhad, J. Recker, M. Weidlich (Eds.), *Business Process Management, LNCS*, vol 9253, pp. 386-405, Springer International Publishing, 2015.
- [5] C. Diamantini, L. Genga, and D. Potena, "Behavioral process mining for unstructured processes", *Journal of Intelligent Information System*, pp 1-28, 2016.
- [6] C. Diamantini, L. Genga, D. Potena, and E. Storti, "Pattern discovery from innovation processes", in: *Collaboration Technologies and Systems (CTS)*, 2013 International Conference on, pp. 457-464, 2013, IEEE.
- [7] C. Diamantini, D. Potena, and E. Storti, "Mining usage patterns from a repository of scientific workflows". In *Proceedings of the 27th Annual ACM Symposium on Applied Computing*, pp. 152-157, ACM, 2012.
- [8] B. van Dongen, and W. van der Aalst, "Multi-phase Process mining: Building Instance Graphs", in : Atzeni, P., Chu, W., Lu, H., Zhou, S., Ling, T. (eds.) *Conceptual Modelling-ER 2004, LNCS*, vol 3288, pp. 362-376, Springer Berlin Heidelberg, 2004
- [9] H. de Man, "Case management: A review of modeling approaches", *BPTrends*, URL: <http://www.bptrends.com/publicationfiles/01-09-ART--20Case%20Management-1-DeMan.%20doc--final.pdf>, January, 2009. Last Access: 16/09/2015.
- [10] M. Rovani, F. Maggi, M. De Leoni, and W. van der Aalst, "Declarative process mining in healthcare". *Expert System with Applications*, 42, , pp 9236-9251, 2015.
- [11] C. Diamantini, L. Genga, D. Potena, and W. van der Aalst, "Building instance graphs for highly variable processes", *Expert System With Applications*, vol 59, pp 101-118, 2016.
- [12] A. Adriansyah, F. Boudewijn, and W. van der Aalst, "Conformance checking using cost-based fitness analysis," in: *15th IEEE International Enterprise Distributed Object Computing Conference*, pp. 55-64, IEEE, 2011.
- [13] B. Van Dongen, et al, "The ProM framework: A new era in process mining tool support.", in: *Applications and Theory of Petri Nets*, pp 444-454, Springer Berlin Heidelberg, 2005.
- [14] W. van der Aalst, T. Weijters, and L. Maruster. "Workflow mining: Discovering process models from event logs.", *Knowledge and Data Engineering, IEEE Transactions on*, vol 16, pp 1128-1142, 2004.
- [15] S. Leemans, D. Fahland, and W. van der Aalst, "Discovering process models from event logs containing infrequent behaviour", in: N. Lohmann, M. Song, P. Wohed (Eds.), *Business Process Management Workshops, LNBIP*, vol 171, pp. 66-78, 2014, Springer International Publishing.
- [16] A. Weijters, W. van der Aalst and A. De Medeiros, "Process mining with the heuristics miner-algorithm," *Technische Universiteit Eindhoven*, Tech. Rep. WP 166, pp 1-34, 2006.